To complete this assessment, we first need to generate the CSV files. Fortunately, all of these datasets are built into R's core datasets library.

Run this initial R script to export the built-in data into the exact CSV files required by your assignment. This will allow the rest of the code to run exactly as requested.

Step 0: Generate the Datasets (Run in R)

R

```
# Generate Nile.csv
write.csv(data.frame(year = time(Nile), value = as.numeric(Nile)), "Nile.csv", row.names = FALSE)

# Generate AirPassengers.csv
write.csv(data.frame(year = time(AirPassengers), passengers = as.numeric(AirPassengers)), "AirPassengers.csv", row.names = FALSE)

# Generate USArrests.csv (State names are captured as row names)
write.csv(USArrests, "USArrests.csv", row.names = TRUE)

# Generate state_x77.csv
write.csv(as.data.frame(state.x77), "state_x77.csv", row.names = TRUE)

# Generate quakes.csv
write.csv(quakes, "quakes.csv", row.names = FALSE)

# Generate volcano_elevation.csv (Convert 2D matrix to long format)
volcano_df <- expand.grid(x = 1:nrow(volcano), y = 1:ncol(volcano))
volcano_df$elevation <- as.vector(volcano)
```

```
write.csv(volcano_df, "volcano_elevation.csv", row.names = FALSE)

print("All CSV files generated successfully in your working directory!")
```

Question 1: Individual Time Series — Smoothing & Detrending (R)

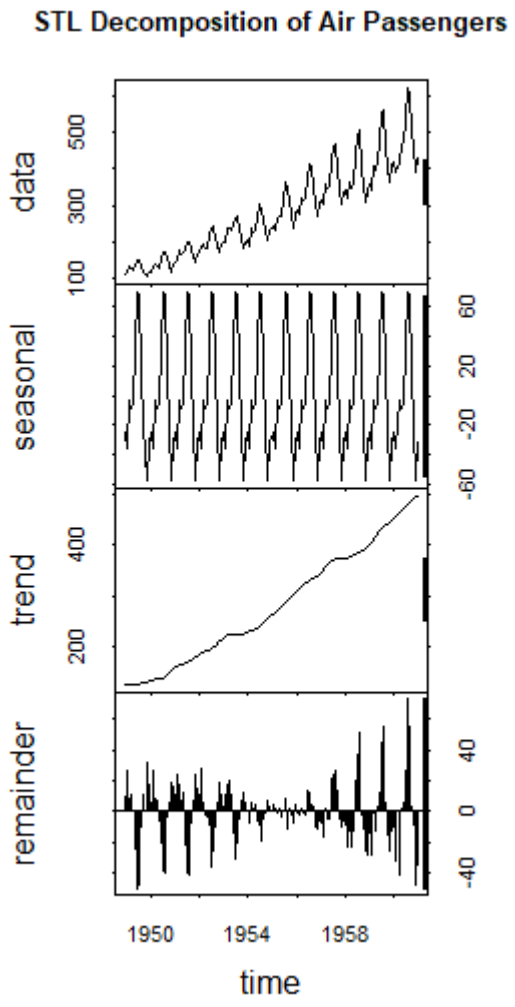
R

```
library(zoo)

# Load and prepare data
nile_df <- read.csv("Nile.csv")
nile_ts <- ts(nile_df$value, start = 1871, frequency = 1)

# Fit linear trend and get residuals
nile_lm <- lm(value ~ year, data = nile_df)
detrended_residuals <- residuals(nile_lm)

# Setup 2-panel plot
par(mfrow = c(2, 1))

# Panel 1: Raw data with 5-year moving average
plot(nile_ts, main = "Nile River Annual Flow (1871-1970)",
    ylab = "Flow", xlab = "Year", col = "gray", lwd = 1.5)
lines(rollmean(nile_ts, k = 5, fill = NA), col = "blue", lwd = 2)
abline(v = 1898, col = "red", lty = 2) # Reference line for visual shift

# Panel 2: Detrended Residuals
plot(nile_df$year, detrended_residuals, type = "l", col = "darkgreen",
    main = "Detrended Residuals", xlab = "Year", ylab = "Residuals")
abline(h = 0, col = "black", lty = 2)
```

par(mfrow = c(1, 1)) # Reset layout

Analysis:

The smoothed series reveals a sharp, permanent downward level shift around 1898–1899. The residual plot confirms this is not just a gradual linear trend; the residuals are consistently positive before 1898 and consistently negative afterward. This step-change corresponds historically to the construction of the first Aswan Low Dam, fundamentally altering the river's flow rather than a natural linear decline.



Question 2: Seasonal Decomposition (STL) & Forecasting (R)

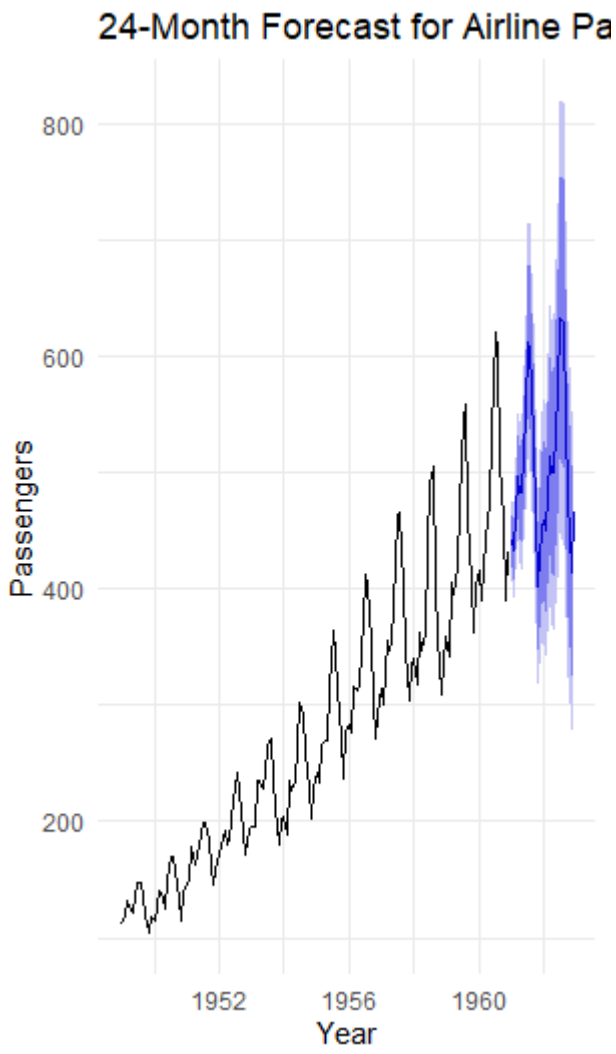
R

library(forecast)

```
# Load and prepare data
air_df <- read.csv("AirPassengers.csv")
air_ts <- ts(air_df$passengers, start = c(1949, 1), frequency = 12)

# Apply STL decomposition
air_stl <- stl(air_ts, s.window = "periodic")
plot(air_stl, main = "STL Decomposition of Air Passengers")

# Fit ETS model and forecast 24 months
air_model <- ets(air_ts) # auto.arima(air_ts) also works
air_forecast <- forecast(air_model, h = 24)

# Plot forecast
autoplot(air_forecast) +
  labs(title = "24-Month Forecast for Airline Passengers",
       y = "Passengers", x = "Year") +
  theme_minimal()
```

Analysis:

A **multiplicative** decomposition is much more appropriate for this series. The raw data plot and the resulting forecast show that as the overall trend (passenger volume) increases over time, the amplitude of the seasonal swings also widens proportionally. An additive model would assume the seasonal effect is constant regardless of the trend level, which contradicts the data. The forecasted seasonal pattern fits perfectly with historical seasonality, successfully capturing the widening summer peaks.



Question 3: Choropleth Map & Interactive Geospatial Plot (R)

R

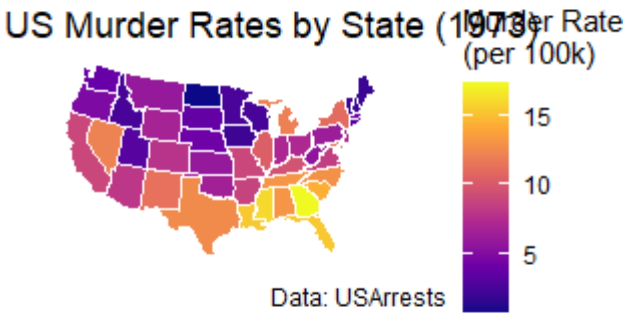
```
library(ggplot2)

library(dplyr)

library(maps)

library(leaflet)


## Part A: Choropleth Map

arrests <- read.csv("USArrests.csv")

# Rename the first column to 'region' and make it lowercase to match map data
```

```
colnames(arrests)[1] <- "region"
arrests$region <- tolower(arrests$region)

state_map <- map_data("state")
map_data_merged <- left_join(state_map, arrests, by = "region")

ggplot(map_data_merged, aes(x = long, y = lat, group = group, fill = Murder)) +
  geom_polygon(color = "white", size = 0.2) +
  scale_fill_viridis_c(option = "plasma", name = "Murder Rate\n(per 100k)") +
  coord_map("albers", lat0 = 39, lat1 = 45) +
  labs(title = "US Murder Rates by State (1973)",
       caption = "Data: USArrests") +
  theme_void()

## Part B: Interactive Geospatial Plot
quakes_df <- read.csv("quakes.csv")

leaflet(quakes_df) %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    lng = ~long, lat = ~lat,
    radius = ~mag, # Size by magnitude
    color = ~colorNumeric("YlOrRd", depth)(depth), # Color by depth
    stroke = FALSE, fillOpacity = 0.7,
    popup = ~paste("Magnitude:", mag, "<br>Depth:", depth, "km")
  ) %>%
  addLegend("bottomright", pal = colorNumeric("YlOrRd", quakes_df$depth),
            values = ~depth, title = "Depth (km)")
```

Analysis:

- **Choropleth:** Georgia exhibits the highest murder rate. There is a distinct regional clustering, with southern states generally showing higher violent crime rates compared to the northern and midwestern states during this period.

- **Interactive Map:** The earthquakes show severe spatial clustering along a distinct arc or fault line (the Tonga Trench off Fiji). The interactive layers reveal that depth gradients follow this geographic curve, visualizing a subduction zone where tectonic plates meet.



Question 4: Cartogram with Map Projections (R)

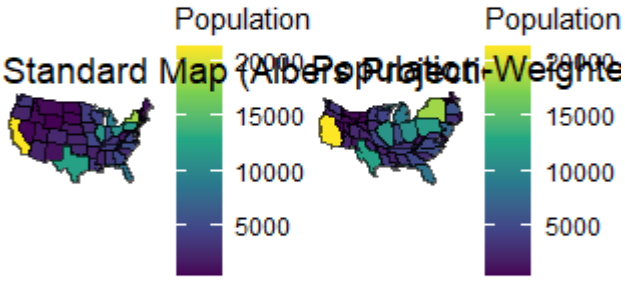
R

```
library(sf)
library(cartogram)
library(ggplot2)
library(dplyr)

# Load data and prepare for joining
state_stats <- read.csv("state_x77.csv")
colnames(state_stats)[1] <- "ID"
state_stats$ID <- tolower(state_stats$ID)

# Get state polygons as an sf object, filter to contiguous US
us_sf <- st_as_sf(maps::map("state", plot = FALSE, fill = TRUE))
us_sf <- left_join(us_sf, state_stats, by = "ID")

# Transform to Albers Equal Area projection (EPSG: 5070) - critical for cartograms
us_sf_albers <- st_transform(us_sf, 5070)

# Generate continuous population-weighted cartogram
us_cartogram <- cartogram_cont(us_sf_albers, weight = "Population", itermax = 5)

# Plot standard map vs cartogram side-by-side
p1 <- ggplot(us_sf_albers) +
 geom_sf(aes(fill = Population)) +
 scale_fill_viridis_c() +
 theme_void() +
 labs(title = "Standard Map (Albers Projection)")
```

```
p2 <- ggplot(us_cartogram) +
 geom_sf(aes(fill = Population)) +
 scale_fill_viridis_c() +
 theme_void() +
 labs(title = "Population-Weighted Cartogram")

library(gridExtra)
grid.arrange(p1, p2, ncol = 2)
```

Analysis:

The choice of an Albers Equal-Area projection is vital for the standard map, as standard longitude/latitude heavily distorts the size of northern states. In the cartogram, physical area is warped to reflect population density: geographically massive but sparsely populated states like Montana and the Dakotas shrink significantly, while smaller, dense states like New Jersey, New York, and Florida swell aggressively. This prevents the "land doesn't vote" cognitive bias common in standard choropleths.



Question 5: 3D Geospatial Heatmap (Python)

Ensure you install the required Python libraries first: pip install pandas plotly.

Python

```
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# 1. Load and prepare the data
df = pd.read_csv("volcano_elevation.csv")
```

```
# Pivot from long format (x, y, elevation) to a 2D matrix (grid)
elevation_grid = df.pivot(index='y', columns='x', values='elevation').values

# 2. Create the Figure with side-by-side subplots (3D and 2D)
fig = make_subplots(
    rows=1, cols=2,
    specs=[[{"type": "surface"}, {"type": "heatmap"}]],
    subplot_titles=("3D Surface Map", "2D Top-Down Heatmap")
)

# Add 3D Surface
fig.add_trace(
    go.Surface(z=elevation_grid, colorscale='Earth', showscale=False,
            hovertemplate="X: %{x}<br>Y: %{y}<br>Elevation: %{z}m<extra></extra>"),
    row=1, col=1
)

# Add 2D Heatmap
fig.add_trace(
    go.Heatmap(z=elevation_grid, colorscale='Earth',
            hovertemplate="X: %{x}<br>Y: %{y}<br>Elevation: %{z}m<extra></extra>"),
    row=1, col=2
)

# Update layout
fig.update_layout(
    title_text="Maunga Whau (Mt Eden) Volcano Topography",
```

```
    height=600,
    width=1100
)

# Adjust 3D axis labels
fig.update_scenes(
    xaxis_title='X Grid',
    yaxis_title='Y Grid',
    zaxis_title='Elevation (m)'
)

fig.show()
```

Analysis:

The 3D surface map instantly communicates the steepness of the volcano's crater walls and the physical depth of the caldera, which are highly intuitive spatial features. The 2D heatmap displays the same maximum and minimum values via color intensity, but it requires the viewer to mentally translate color gradients into vertical height, making it harder to rapidly grasp the terrain's physical concavity.